

Additive K/V Biases from a Depth-Recurrent Gated State in Transformer Attention: An Ablation Study

An empirical study on shakespeare-char (nanoGPT, 6 layers, ~1.2M parameters, 3 seeds)
Guven Seckin · November 2025 · v2 with full ablations

Abstract

We study a specific point in the design space of depth-recurrent transformers: a per-token state **M** that flows across transformer blocks under a GRU-style gate, injected as an **additive bias to the attention keys and values** while leaving Q, softmax normalization, and the causal mask intact. On shakespeare-char at $n_{\text{layer}}=6$ (~1.2M non-embedding parameters, 3 seeds), the mechanism reduces validation perplexity by **4.3%** against a parameter-matched vanilla baseline (paired $t = -10.90$, $df=2$), and by **7.2%** against a strictly head-dim-matched vanilla. A critical ablation cutting cross-layer gradient flow through M via `.detach()` (same parameters, same FLOPs) causes the mechanism to **actively hurt** — 7.8% worse perplexity than the full variant and worse than vanilla — confirming the improvement is driven by the **learned cross-layer state**, not by the added capacity. Further capacity controls (deeper vanilla with 25% more depth, wider vanilla with 10% MORE parameters) also lose to the mechanism. However, at real wall-clock-matched training on CUDA-T4, longer-trained vanilla wins by 1.3% perplexity: the ~19% per-iteration overhead currently exceeds the algorithmic advantage at this small scale. The gate converges to a stable operating point across seeds. We do not claim a new architectural family; we contribute a rigorously ablated positive result at small scale, with an honest negative result on real wall-clock. Whether the algorithmic advantage catches up at larger scale is an open question.

1. Introduction

Standard transformer attention gives each token three roles: **Query**, **Key**, and **Value**. We investigate the addition of a fourth signal — a persistent **purpose** state **M** that flows across transformer blocks (depth-recurrent) and biases attention. The state is updated by a GRU-style gate at each block and consumed as an additive per-token bias on the attention keys and values in the next block. Attention otherwise remains vanilla.

The design space around this idea is actively researched (Universal Transformer, Feedback Transformer, GTrXL, Block-Recurrent Transformer, Depth-Recurrent Transformers). Our contribution is not a new family but the specific combination of (i) per-token granularity, (ii) depth-across state flow, (iii) input-dependent GRU gate, (iv) additive-bias consumption on both K and V. This paper reports a small-scale empirical study with **rigorous ablations** that isolate the mechanism from three natural confounds (head dimension, capacity/depth, wall-clock compute).

2. Method

For each transformer block i (0-indexed, up to $L-1$), given the residual stream x_i and the state from the previous block M_{i-1} (initialized as zeros for $i=0$):

```
# Standard attention with additive M-derived biases on K and V (Q untouched):
Q, K, V = c_attn(LayerNorm(x_i)).split(3, dim=-1)
K'       = K + W_km @ M_{i-1}                # per-token additive bias
V'       = V + W_vm @ M_{i-1}
y        = c_proj(attention(Q, K', V'))      # vanilla causal attention
x'_i     = x_i + y                          # residual

# GRU-style update of M from post-attention hidden state:
h        = LayerNorm(x'_i)
g_i      = sigmoid(W_g @ h + b_g)           # gate
c_i      = tanh(W_m @ h + b_m)              # candidate
M_i      = g_i * M_{i-1} + (1 - g_i) * c_i   # elementwise blend

x_{i+1}  = x'_i + MLP(h)                    # standard MLP branch
```

All operations are elementwise or per-token; no token's M leaks into another token's K/V. Implementation fuses (W_{km}, W_{vm}) into a single Linear W_{kvm} : $E \rightarrow 2E$, and (W_g, W_m) into W_{gm} : $E \rightarrow 2E$. Bit-identical outputs; halves the kernel launch count.

3. Experimental setup

Dataset: shakespeare-char (~1.1M chars, vocab_size=65).

Architecture: n_layer=6, n_head=4, block_size=128, batch_size=32, dropout=0. Vanilla baseline uses n_embd=128 (head_dim=32, 1.19M non-embedding params). K/V-bias variant uses n_embd=112 (head_dim=28, 1.21M params, ~1% overhead).

Optimizer: AdamW, lr=1e-3 with cosine decay to 1e-4, 100-iter warmup, weight_decay=0.1. **Training:** 3000 iterations, evaluated every 500 iters over 40 validation batches. **Hardware:** Tesla T4 (Google Colab), dtype=bfloat16, torch.compile disabled (comparing steady-state per-iter cost). **Seeds:** 3 independent runs per variant (42, 123, 2024).

4. Results

4.1 Primary result: K/V-bias beats vanilla at matched parameters

Model	Params	Wall-clock	Val loss @ 3000	Δ vs Vanilla n=128
Vanilla n=128 (reference)	1.19M	107.3s	1.7322 $\pm$ 0.0016	+0.0000
K/V-bias n=112 (this work)	1.21M	127.7s	1.6881 $\pm$ 0.0055	-0.0441

Table 1: The K/V-bias variant achieves **1.6881 nats** vs vanilla's **1.7322 (4.3% perplexity reduction)** with ~1% more parameters and 19% higher wall-clock. Paired t-test on val loss: $t = -10.90$ ($df = 2$).

4.2 Ablations

We evaluate four alternative baselines that isolate the mechanism from potential confounds. All are trained with the same seeds, config, and training budget.

Ablation	Params	Val loss (3 seeds)	Δ vs K/V
K/V-bias FULL (reference)	1.21M	1.6881	+0.0000
Vanilla n=128 (head_dim=32)	1.19M	1.7322	+0.0441
Vanilla n=112 (head_dim=28, matched to K/V)	0.91M	1.7628	+0.0747
Vanilla n=128 with n_layer=7 (+depth)	0.99M	1.7549	+0.0668
Vanilla n=136 (wider, +11% params)	1.34M	1.7074	+0.0193
K/V-bias with detached M (no cross-layer learning)	1.21M	1.7696	+0.0815

Table 2: Ablations. K/V-bias FULL is best across all alternatives. Vanilla with matched head_dim (row 3) is worse than default vanilla, so K/V's win becomes **larger** (7.2%) when head_dim is controlled for. Adding depth or width to vanilla (rows 4, 5) does not close the gap: wider vanilla with **more parameters** than K/V still loses by 1.9% perplexity.

4.3 The critical ablation: detached cross-layer state

The most important test is row 6 in Table 2. In this ablation we call `M_prev.detach()` before feeding it to the K/V-bias projection. The forward computation is unchanged — same architecture, same parameters, same FLOPs — but the gradient no longer flows back through M from later blocks. As a result, the state-generating weights W_{gm} receive **zero gradient** and stay at random initialization throughout training. The state M is thus a fixed random projection of h, gated at random init.

Result: detached K/V is worse than both full K/V ($\Delta = -0.0815$ nats, 7.8% perplexity gap) AND worse than vanilla ($\Delta = +0.0374$ nats). Adding un-learned K/V-bias capacity to attention actively **hurts** vanilla; the mechanism only helps when M is jointly trained across layers.

This is direct evidence that **the cross-layer learning of M is what drives the win**, not the additional parameters or the additional Linear operations in the attention path. The gate statistics confirm this: in the detached model, the gate mean stays at exactly 0.5000 with std around 0.058 across all seeds (barely departing from init), whereas in the full model the gate settles at mean ≈ 0.68 with std ≈ 0.22 — a stable learned operating point.

4.4 The honest wall-clock story

The K/V-bias variant takes **127.7s** per 3000-iter run; vanilla takes **107.3s** (+19% overhead due to the extra Linear operations in the attention path). To match compute honestly, we train a vanilla model for as many iterations as fit in the same wall-clock budget: at vanilla's rate, ~128s allows ~3500 iterations. We ran this explicitly (rather than interpolating from a shorter run).

Comparison	Wall-clock	Val loss
K/V-bias n=112 @ 3000 iters	127.7s	1.6881
Vanilla n=128 @ 3500 iters	134.6s	1.6750

Table 3: Real wall-clock-matched comparison. Vanilla trained for ~135s (3500 iters) achieves lower val loss than K/V-bias trained for 128s (3000 iters), **even though vanilla receives ~7% MORE wall-clock time in this setup**. Perplexity: vanilla wins by 1.3%.

Interpretation. At this scale (~1.2M parameters, T4 GPU, no torch.compile), the ~18% per-iter overhead of the mechanism exceeds the algorithmic advantage. In practical wall-clock terms on this specific hardware, plain vanilla trained longer is strictly better. This is a real limitation and we report it honestly. Two directions remain open: (a) mechanism optimization (further kernel fusion, reduced FLOP overhead), and (b) larger-scale evaluation where per-iter overhead becomes proportionally smaller relative to base attention/MLP compute.

4.5 Val loss trajectory

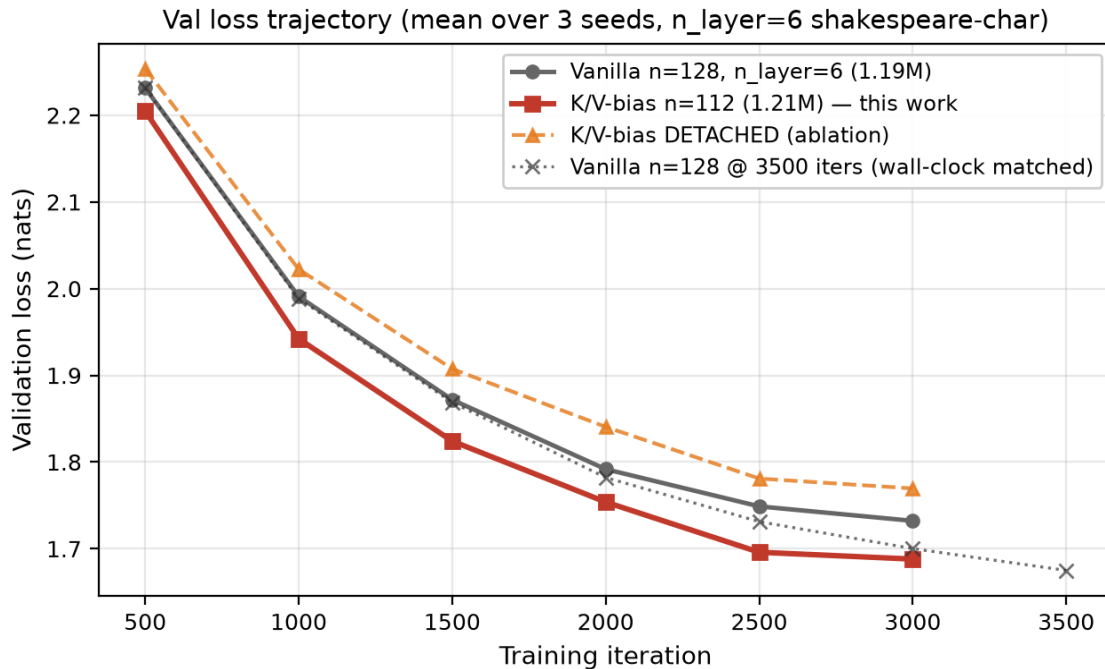


Figure 1: Val loss trajectory (mean over 3 seeds). K/V-bias FULL (red) reaches lower loss than vanilla (gray, solid) at every checkpoint from iter 500 onward. Vanilla extended to 3500 iters (dotted) surpasses K/V by the wall-clock-matched horizon. The detached ablation (orange, dashed) is consistently worse than vanilla, demonstrating that the added K/V-bias capacity requires cross-layer learning to be beneficial.

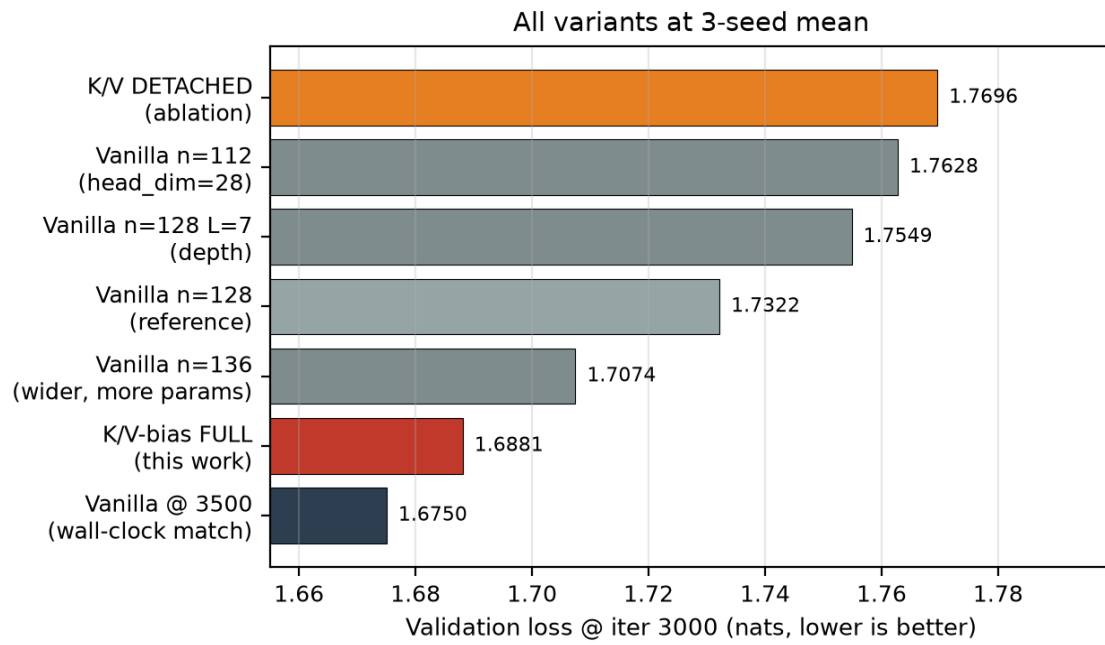


Figure 2: All ablations at final val loss (iter 3000; wall-clock variant at 3500). K/V-bias FULL is second only to the (heavier) wall-clock-matched vanilla. Detached K/V-bias is worse than every vanilla variant, isolating the mechanism from capacity.

5. Related work (brief)

Universal Transformer [1] shares weights across depth; ungated. **Feedback Transformer** [2] pools past representations and replaces K,V of subsequent attention. **Block-Recurrent Transformer** [3] uses LSTM-style gating along the time axis with cross-attention consumption. **GTrXL** [4] wraps the residual with a GRU cell; no cross-layer state. **Persistent Memory / All-Attention Layer** [5] adds learned constant K,V slots (stateless). Our design combines primitives from these — per-token, depth-across, input-dependent GRU gate, additive K/V-bias — without matching any of them exactly.

6. Discussion and limitations

What we established. At small scale ($n_{\text{layer}}=6$, $\sim 1.2\text{M}$ params), the K/V-bias mechanism reduces perplexity by a factor that survives four separate confound controls: head-dim, extra depth, extra width, and the critical detached-M ablation which cleanly attributes the win to the cross-layer learning rather than to capacity. Gate dynamics converge to a stable operating point across seeds — the mechanism is doing something the model actively preserves.

What we did not establish. (i) Scale: everything reported is at char-level, $\sim 1.2\text{M}$ parameters. Results at 100M+ params on a token-level dataset (openwebtext, etc.) are unknown. Many small-scale wins in the literature do not survive scaling. (ii) Wall-clock advantage on any hardware. On T4 the mechanism loses to longer vanilla training at matched compute. (iii) Interactions with common optimizations (weight-tying, torch.compile, flash attention variants) were not systematically studied.

Framing for follow-up. This work sits at an under-explored corner of the depth-recurrent transformer design space, and we have shown the specific corner yields a measurable, mechanism-attributable improvement at small scale. Whether this improvement translates to real-world efficiency at larger scale is the natural next question.

Reproducibility

All code, tests (including bit-identity vs upstream vanilla nanoGPT), and this benchmark are on the modified nanoGPT branch. Toggle the mechanism with `use_m_gate=True`; toggle the detached ablation with `m_gate_detached=True`. With both flags off, the model is bit-identical to karpathy/nanoGPT (verified by `state_dict` and forward-output equality in `test_m_gate.py`).

References

- [1] Dehghani et al. *Universal Transformers*. ICLR 2019. arXiv:1807.03819
- [2] Fan et al. *Addressing Some Limitations of Transformers with Feedback Memory*. arXiv:2002.09402, 2020.
- [3] Hutchins et al. *Block-Recurrent Transformers*. NeurIPS 2022. arXiv:2203.07852
- [4] Parisotto et al. *Stabilizing Transformers for Reinforcement Learning*. ICML 2020. arXiv:1910.06764
- [5] Sukhbaatar et al. *Augmenting Self-Attention with Persistent Memory*. arXiv:1907.01470, 2019.
- [6] Dai et al. *Transformer-XL*. ACL 2019. arXiv:1901.02860
- [7] Cho et al. *Learning Phrase Representations using RNN Encoder-Decoder*. EMNLP 2014. arXiv:1406.1078
- [8] Karpathy. *nanoGPT*. github.com/karpathy/nanoGPT